

ENGINEERING PLANNING DOSSIER

LIFEOS

Software Requirements, Product Scope, Technical Research,
Database Design, Architecture, API Planning, AI Agent Design,
Security, Testing and Development Readiness

**Intent -> Understand -> Research -> Generate Plan -> Execute -> Track -> Decide
What To Do Next**

DOCUMENT

Pre-Development Baseline v1.0

DATE

28 Sep 2026

BACKEND

Python + FastAPI + PostgreSQL

AI / MAPS

**Gemini + ADK + Google Maps
Platform**

Purpose: establish the design baseline before feature development. This document is intended to be the single planning reference for the first production-style implementation of LIFEOS.

Current local foundation already verified: Python 3.12 + uv, FastAPI/Uvicorn, PostgreSQL 16, SQLAlchemy async, asyncpg, Alembic and database connectivity.

1. Executive Summary

LIFEOS is an AI-powered real-world activity planner that turns a user's natural-language intention into a context-aware action plan.

Core product promise: LIFEOS should reduce the gap between "I need to do this" and "I know exactly what to do next".

The product is deliberately different from a generic todo manager. The important capability is inference: a user supplies intent and relevant context, and the system derives preparation, tasks, checkpoints, route information, packing needs, itinerary structure and contextual recommendations. The output must be grounded in real-world data where claims depend on current facts.

Primary design principle

AI reasons.
Tools provide reality.
Backend validates and persists.
The user remains in control of external actions.

Planning outcome

What is already done

Local application/server foundation and PostgreSQL/Alembic infrastructure are in place and tested.

What this dossier freezes

Product scope, SRS, domain entities, relationships, ER model, module boundaries, API shape, AI workflow, tool contracts and non-functional requirements.

What comes next

Database models and migrations, then schemas, repositories, services and routes, followed by the first end-to-end AI vertical slice.

What is intentionally postponed

Microservices, Redis/Celery, booking/transactional automation, broad autonomous actions, vector search and Kubernetes.

2. Table of Contents

Section	Content
3	Product Definition and Vision
4	Software Requirements Specification (SRS)
5	Functional Requirements and Use Cases
6	Non-Functional Requirements
7	Technical Research: AI, Agents and Gemini
8	Technical Research: Maps, Places, Auth and Cloud
9	Architecture Decisions
10	Backend Directory / Module Plan
11	Domain Model and Database Strategy
12	ER Diagram
13	Relational Relationships and Constraints
14	Database Data Dictionary
15	API and Request Flow Plan
16	AI Agent Architecture and Tool Contracts
17	Grounding, Evidence and Verification
18	Security and Trust Boundaries
19	Reliability, Errors, Retries and Idempotency
20	Testing and Evaluation Strategy
21	Observability and Operations
22	Deployment Plan
23	Development Roadmap
24	Definition of Ready / Done
25	Pre-Development Checklist
26	Open Decisions and Future Expansion
27	Reference Sources

3. Product Definition and Vision

3.1 Product statement

"LIFEOS turns any real-world intention into a context-aware action plan."

3.2 Core user loop

```
User intent
|
v
Understand activity context
|
v
Research current / location-sensitive facts
|
v
Generate structured plan
|
+--> Sections
+--> Tasks / subtasks / dependencies
+--> Route / stops
+--> Places / recommendations
+--> Packing
+--> Itinerary
|
v
Track progress
|
v
"What should I do next?"
```

3.3 Signature behavior

The system should infer implications without pretending certainty. For travel it can derive preparation and route context. For appointments or institutional visits it can structure timing, travel buffers and user-provided requirements without inventing unsupported rules.

3.4 Product boundaries

In scope

- Intent capture and structured activity understanding
- AI plan generation
- Task/checklist management
- Maps and route context
- Contextual places and recommendations
- Packing and itinerary planning
- Progress, next action and run traceability

Out of scope for MVP

- Automatic booking or payment
- Autonomous messaging to third parties
- Medical/legal advice
- Unbounded autonomous tool access
- Microservice decomposition
- Enterprise multi-tenancy
- Advanced collaborative editing

4. Software Requirements Specification (SRS)

4.1 Purpose

This SRS defines the functional and non-functional requirements for the initial LIFEOS system, with emphasis on the activity-to-plan workflow and the trust boundary between generated suggestions and verified real-world facts.

4.2 Users and stakeholders

Actor	Need	System responsibility
End user	Turn an intention into an actionable plan	Capture intent, show plan, allow edits, track progress and explain next action
Authenticated user	Own and revisit personal plans	Enforce user ownership and persist history
System administrator / developer	Operate the application safely	Observe runs, errors, tool calls, usage and deployment health
External providers	Return current location / AI data	Use narrow integrations and provider-compatible policies

4.3 Core requirements

ID	Requirement	Priority
FR-01	User can create an activity from natural-language intent and optional structured constraints.	MUST
FR-02	System converts raw intent into structured activity context.	MUST
FR-03	System generates a plan containing sections and actionable tasks.	MUST
FR-04	Tasks support priorities, status, due times and subtasks.	MUST
FR-05	Tasks can express dependencies without self-dependencies or cycles.	MUST
FR-06	System can attach route and place context to a plan.	SHOULD
FR-07	System can generate packing and itinerary items where relevant.	SHOULD
FR-08	Every generation run is traceable through a plan run record.	MUST
FR-09	Current/fresh factual claims can retain evidence metadata.	MUST
FR-10	User can view progress and the next actionable task.	MUST
FR-11	External side effects require explicit approval.	SHOULD
FR-12	Generation failures are visible and recoverable without corrupting the plan.	MUST

5. Functional Requirements and Use Cases

5.1 Primary use case - Generate a plan

```
Given a user provides an activity intent
When the user requests planning
Then LIFEOS should:
1. authenticate the user
2. load the activity and constraints
3. understand the activity type and missing context
4. research current facts where needed
5. use maps/places when location matters
6. produce a structured plan draft
7. verify important claims and structure
8. persist the plan, tasks and supporting data
9. expose generation status and progress
```

5.2 Example: travel

Input: "I am going to Manali from Delhi for 4 days by car."

Expected system behavior: infer the trip structure; calculate route information; identify reasonable route-stop categories such as fuel, food and rest where useful; organize destination activities; generate a prioritized packing list; build an itinerary; create actionable tasks with sensible dependencies; surface current factual sources for time-sensitive claims.

5.3 Example: hospital visit

Input: "I have a hospital consultation tomorrow." The system can structure appointment timing, travel buffer, user-provided documents/reports, arrival steps and post-visit administrative reminders. It should not invent medical requirements or provide clinical judgment.

5.4 Edge cases to design before implementation

Case	Expected behavior
Ambiguous intent	Ask targeted clarifying question or generate a conservative plan marked with assumptions.
Missing destination	Do not invent it. Keep planning at generic activity level until clarified.
Provider timeout	Retry with backoff; continue with partial plan only when safe; label missing external data.
Conflicting facts	Prefer stronger/current source; preserve evidence and mark uncertainty when unresolved.
Duplicate create request	Use idempotency semantics to avoid duplicate activities/runs.
Regeneration	Create a new plan generation run; avoid destructive overwrite until new plan is verified.
Task dependency cycle	Reject at persistence/validation stage.
User deletes activity during run	Cancel or mark run stale; do not resurrect deleted data.

6. Non-Functional Requirements

Area	Requirement / target
Performance	Normal CRUD endpoints should be fast and predictable; long AI generation should be asynchronous from the user's perspective.
Availability	Stateless API instances; durable state in PostgreSQL; provider failures should degrade gracefully.
Security	Firebase ID token verification, HTTPS, least-privilege service accounts, secret management, input validation and strict tool allowlists.
Correctness	Generated structure must validate against schemas and relational constraints before persistence.
Grounding	Current facts should be backed by provider/search evidence. AI inference must be distinguishable from external facts.
Maintainability	Clear module boundaries: API, schemas, services, repositories, models, agents, integrations and core.
Observability	Structured logs, request IDs, plan-run IDs, tool-call status and timing.
Testability	Unit tests for domain logic; integration tests for DB/API; evaluation dataset for agent behavior.
Portability	Local Docker-compatible deployment path and Cloud Run-compatible container.
Data lifecycle	Define retention/deletion for plans, evidence and tool logs before production.

Design target: production-grade engineering quality comes from explicit boundaries, validation, observability and testability - not from creating microservices prematurely.

7. Technical Research - AI, Agents and Gemini

7.1 Current Gemini model research

As of September 2026, Google's Gemini documentation lists `gemini-3.8-flash` as a stable model. Google describes it as its most intelligent Flash model and specifically positions it for long-horizon software engineering, autonomous agents and complex enterprise workflows. The model page lists structured outputs, function calling, search grounding and Google Maps grounding among supported capabilities. (R1, R2)

Planning decision: freeze the model behind configuration, e.g. `GEMINI_MODEL=gemini-3.8-flash`, rather than scattering the model ID through code.

7.2 Interactions API

Google's current documentation states that the Interactions API became generally available in June 2026 and is recommended for new projects. It provides a unified interface for models and agents, with capabilities including structured outputs, tool orchestration, optional server-side state and background execution. (R3)

For LIFEOS, this supports the plan-run concept: a long-running plan generation can be represented as a tracked interaction/run rather than forcing a long synchronous HTTP request.

7.3 Agent Development Kit

Google's ADK documentation remains centered on building, evaluating and deploying agents. Current ADK material also emphasizes structured agent development, tools, evaluation, observability and deployment. Google's 2026 multi-agent guidance describes sequential pipelines as a deterministic orchestration pattern that is relatively easy to debug. (R4, R5)

Use ADK for

Agent definition, orchestration, tool execution, session/runtime integration and evaluation workflows.

Do not use the model for

Authorization, arbitrary database writes, invariant enforcement or replacing deterministic route/data providers.

7.4 AI architecture decision

```
FastAPI service
```

```
|
+--> Plan Orchestrator
    |
    +--> Activity Understanding Agent
    +--> Research Agent
    +--> Maps Agent
    +--> Recommendation Agent
    +--> Planner Agent
    +--> Verification Agent
```

Agents call narrow tools. The application layer persists validated results.

8. Technical Research - Maps, Places, Auth and Cloud

8.1 Routes API

Google Maps Routes API currently exposes Compute Routes and Compute Route Matrix. Compute Routes accepts origin, destination, optional intermediate waypoints and travel options; Compute Route Matrix evaluates origin/destination combinations. Both APIs require response field masks, which is important for latency, payload size and cost control. (R6, R7, R8)

Tool design: wrap provider APIs in small functions such as `compute_route()` and `compute_route_matrix()`. Do not expose a single giant "plan trip" tool.

8.2 Places API (New)

Google's documentation identifies Places API (New) as the current version and exposes Text Search, Nearby Search, Place Details, Place Photos and Autocomplete. The service uses explicit field masks; the docs state that a field mask is required for search responses. (R9, R10)

For LIFEOS, cache provider identifiers and only request fields actually needed for the UI or planner. A place ID should be treated as a provider identifier, not as LIFEOS's internal primary key.

8.3 Firebase Authentication

Firebase Authentication can provide sign-in, while a custom backend verifies the Firebase ID token and extracts the user's `uid`. Firebase's current server documentation explicitly describes this pattern. (R11)

8.4 Cloud Run

Cloud Run automatically scales service revisions and can scale to zero by default. Its service request timeout is configurable up to 60 minutes. The container must listen on the platform-provided port and on `0.0.0.0`. (R12, R13)

For LIFEOS, regular API endpoints should remain short. Long agent runs should use asynchronous execution patterns and a durable run record rather than relying on an open HTTP request for the full planning workflow.

8.5 Cloud SQL

Google documents direct Cloud Run integration with Cloud SQL for PostgreSQL, including the supported connection patterns and Cloud SQL integration configuration. (R14)

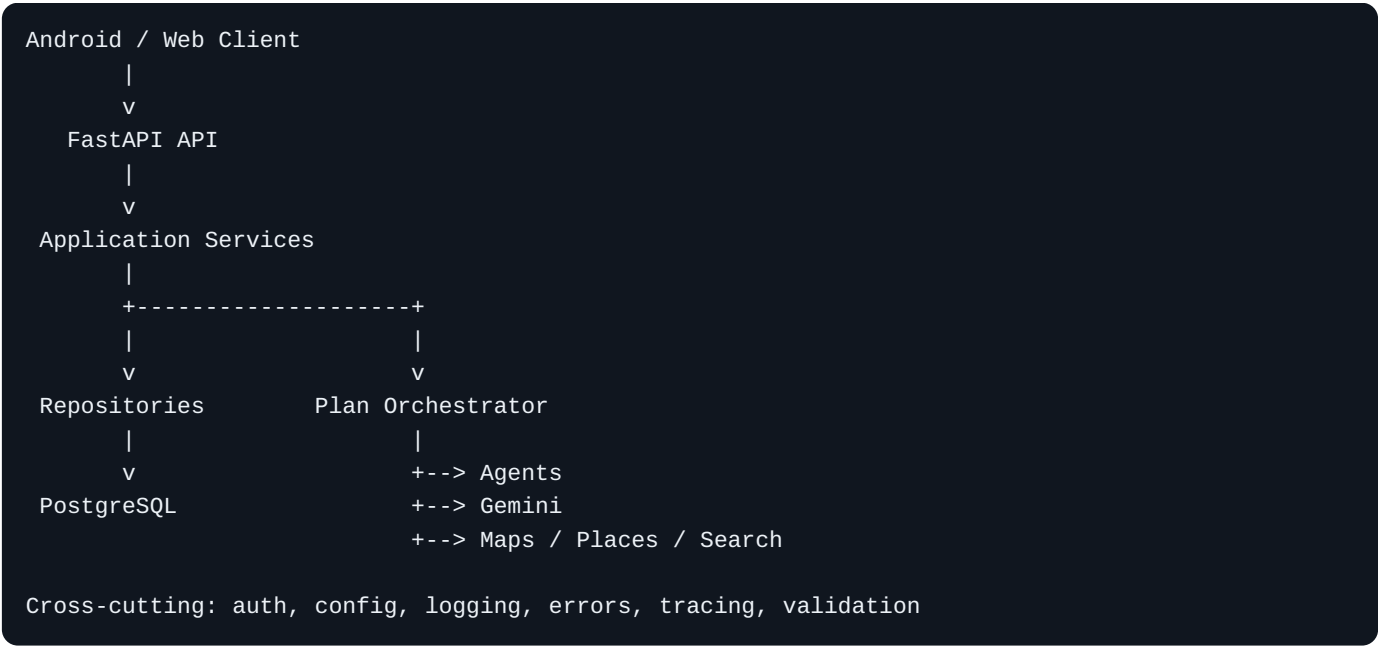
9. Architecture Decisions

9.1 Architectural style

Modular monolith + agent runtime for MVP

The backend should be one deployable application with strict internal module boundaries. This keeps the hackathon implementation fast and debuggable while preserving the option to extract an AI worker later if scaling or team boundaries justify it.

9.2 Component architecture



9.3 Boundary rules

Layer	Owns	Must not own
API routes	HTTP, auth dependencies, request/response mapping	Complex business logic
Schemas	Input/output validation	Persistence
Services	Business/application workflows	Raw HTTP concerns
Repositories	Database access	AI reasoning
Models	Persistence mapping and relational constraints	HTTP behavior
Agents	Reasoning and orchestration decisions	Authorization and unrestricted DB mutation
Integrations	Provider-specific HTTP/client logic	Business-level orchestration

9.4 Controller naming

In FastAPI, the `api/v1/*.py` route modules act as the controller layer. A separate controller package is unnecessary unless the API layer becomes large enough to justify it.

10. Backend Directory / Module Plan

```
backend/
├─ alembic/versions/
├─ src/lifeos/
│   └─ main.py
│   └─ core/ -> config.py, exceptions.py, logging.py, security.py
│   └─ db/ -> base.py, session.py, models/
│       │   models = user, activity, plan, section, task, place, route,
│       │               packing_item, itinerary_item, plan_run, tool_call, evidence
│   └─ api/v1/ -> router.py, activities.py, plans.py, tasks.py, places.py, routes.py
│   └─ schemas/ -> activity.py, plan.py, task.py, place.py, common.py
│   └─ repositories/ -> activity.py, plan.py, task.py, place.py
│   └─ services/ -> activity_service.py, plan_service.py, task_service.py,
next_action_service.py
│   └─ agents/ -> orchestrator.py, activity_understanding.py, research.py, maps.py,
recommendations.py, planner.py, verifier.py
│   └─ integrations/ -> gemini.py, maps.py, places.py, search.py
└─ tests/ -> unit/, integration/, evaluations/
```

10.1 Rules

- Imports should point inward toward stable abstractions where possible.
- Provider SDK details stay inside `integrations/`.
- Database access goes through repositories or a clearly justified service-level transaction function.
- Agents return structured objects, not ad hoc dictionaries scattered across the codebase.
- Business rules belong in services/domain validation, not route functions.

11. Domain Model and Database Strategy

11.1 Entity inventory

Entity	Purpose	Why it exists
User	Application owner	Identity and ownership boundary
Activity	Original real-world intention	Stable source context across regenerated plans
Plan	Generated actionable plan snapshot	Allows multiple generations and history
PlanSection	Logical grouping of plan items	UI sections such as Before You Leave, Journey, Destination
Task	Actionable work item	Checklist, subtasks and next-action logic
TaskDependency	Task-to-task prerequisite	Supports ordering and readiness
Place	External place record	Provider place identity and normalized location data
PlanPlace	Plan-place association	Resolves N:M and stores contextual role
Route	Calculated journey	Stores route snapshot / summary
RouteStop	Ordered route checkpoint	Represents stops and detour context
PackngItem	Item to carry	Priority-aware preparation
ItineraryItem	Scheduled activity	Time-based plan segment
PlanRun	Generation execution record	Status, retries, model, timestamps, traceability
ToolCall	External call log	Reliability, debugging, cost and evidence tracking
Evidence	Source/claim record	Grounding and trust layer

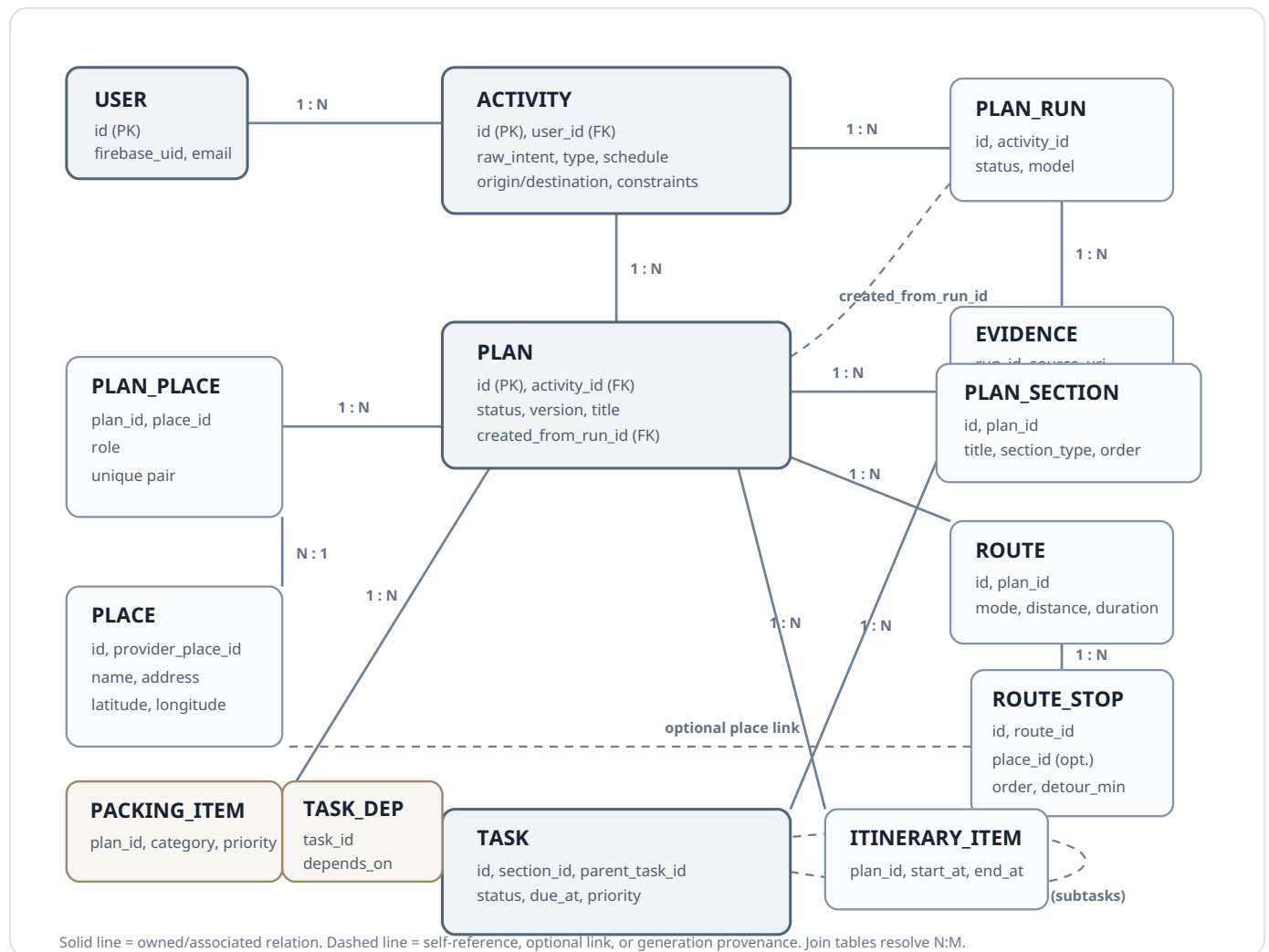
11.2 JSONB usage

Use relational columns for stable, query-critical fields and JSONB for genuinely evolving context produced by AI. Example: `activities.constraints_json` can store flexible preferences while fields such as `start_at` , `end_at` , `origin_text` and `destination_text` remain first-class columns.

Do not put the whole database in JSONB. Tasks, ownership, statuses, dependencies, route relationships and evidence references need relational integrity and indexes.

12. ER Diagram

The diagram below is the proposed conceptual relational model for the first production-style LIFEOS backend.



Relationship summary

- User 1:N Activity
- Activity 1:N Plan
- Plan 1:N PlanSection and 1:N auxiliary plan items
- PlanSection 1:N Task
- Task 1:N Task via parent_task_id for subtasks
- Task N:M Task via TaskDependency for prerequisites
- Plan N:M Place via PlanPlace
- Plan 1:N Route and Route 1:N RouteStop
- Plan 1:N PackingItem and 1:N ItineraryItem
- Activity 1:N PlanRun; PlanRun 1:N ToolCall and Evidence

13. Relational Relationships and Constraints

Relationship	Cardinality	Key rule
User -> Activity	1:N	<code>activities.user_id</code> NOT NULL; delete policy should be explicit.
Activity -> Plan	1:N	Regeneration creates a new plan snapshot rather than destructive overwrite.
Plan -> Section	1:N	Unique <code>(plan_id, order_index)</code> .
Section -> Task	1:N	Each task belongs to one plan section.
Task -> Task	1:N self	<code>parent_task_id</code> nullable for top-level tasks.
TaskDependency	N:M self	Unique pair; reject self-loop; validate cycles in service layer.
Plan -> Place	N:M	Join table stores contextual role such as destination, food, fuel, rest, attraction.
Route -> RouteStop	1:N	Unique ordered index; optional place for generic checkpoints.
Plan -> Itinerary	1:N	Time ranges and ordering should not silently overlap unless allowed.
PlanRun -> ToolCall	1:N	Every provider call is traceable to a run.

13.1 Status enums

Entity	Suggested values
Activity	DRAFT, ACTIVE, COMPLETED, ARCHIVED
Plan	GENERATING, READY, FAILED, ARCHIVED
Task	TODO, IN_PROGRESS, COMPLETED, BLOCKED, CANCELLED
PlanRun	QUEUED, RUNNING, SUCCEEDED, FAILED, CANCELLED, SUPERSEDED
ToolCall	STARTED, SUCCEEDED, FAILED, TIMEOUT

14. Database Data Dictionary

14.1 users

Column	Type	Rules	Meaning
id	UUID	PK	Internal LIFEOS identity
firebase_uid	VARCHAR	UNIQUE, NOT NULL	Firebase user identifier
email	VARCHAR	UNIQUE, NOT NULL	Display / account email
display_name	VARCHAR	NULL	Optional name
created_at	TIMESTAMPTZ	NOT NULL	Creation time
updated_at	TIMESTAMPTZ	NOT NULL	Last update time

14.2 activities

Column	Type	Rules	Meaning
id	UUID	PK	Activity identifier
user_id	UUID	FK users.id	Owner
title	VARCHAR	NOT NULL	Human-readable activity title
raw_intent	TEXT	NOT NULL	Original user input
activity_type	VARCHAR	NOT NULL	Travel, appointment, errand, interview, relocation, etc.
start_at / end_at	TIMESTAMPTZ	NULL	Known time window
origin_text / destination_text	TEXT	NULL	Human location inputs
travel_mode	VARCHAR	NULL	DRIVE, TRANSIT, WALK, etc.
constraints_json	JSONB	NULL	Flexible constraints and preferences
created_at / updated_at	TIMESTAMPTZ	NOT NULL	Audit timestamps

14.3 plans and plan_sections

plans: id, activity_id, title, status, version, created_from_run_id, created_at, updated_at.

plan_sections: id, plan_id, title, description, section_type, order_index, created_at, updated_at.

14. Database Data Dictionary - Continued

14.4 tasks

Recommended columns: `id`, `plan_section_id`, `parent_task_id`, `title`, `description`, `status`, `priority`, `due_at`, `estimated_minutes`, `source`, `metadata_json`, `created_at`, `updated_at`, `completed_at`.

14.5 places

Recommended columns: `id`, `provider`, `provider_place_id`, `name`, `formatted_address`, `latitude`, `longitude`, `primary_type`, `types_json`, `rating`, `website_uri`, `phone`, `opening_hours_json`, `photo_refs_json`, `raw_metadata_json`, `created_at`, `updated_at`.

14.6 routes and route_stops

routes: `id`, `plan_id`, `provider`, `travel_mode`, `origin_text`, `destination_text`, `distance_meters`, `duration_seconds`, `polyline`, `route_metadata_json`, `timestamps`.

route_stops: `id`, `route_id`, `place_id` nullable, `label`, `order_index`, `detour_minutes`, `reason`, `arrival_estimate`, `metadata_json`.

14.7 packing and itinerary

packing_items: `plan_id`, `item`, `category`, `priority` (ESSENTIAL/RECOMMENDED/OPTIONAL), `quantity`, `reason`, `checked_at`.

itinerary_items: `plan_id`, `title`, `description`, `start_at`, `end_at`, `place_id` nullable, `order_index`, `status`, `task_id` nullable.

14.8 generation trace

plan_runs: `id`, `activity_id`, `plan_id` nullable, `status`, `model`, `prompt_version`, `started_at`, `finished_at`, `error_code`, `error_message`, `metrics_json`.

tool_calls: `id`, `plan_run_id`, `tool_name`, `provider`, `status`, `request_hash`, `started_at`, `finished_at`, `latency_ms`, `response_summary_json`, `error_code`.

evidence: `id`, `plan_run_id`, `source_type`, `source_uri`, `title`, `retrieved_at`, `claim`, `relevance`, `metadata_json`.

15. API and Request Flow Plan

15.1 API namespace

/api/v1

15.2 Core endpoints

Method	Endpoint	Purpose
POST	/activities	Create activity from intent
GET	/activities	List user's activities
GET	/activities/{id}	Get one activity
PATCH	/activities/{id}	Edit activity context
POST	/activities/{id}/plan-runs	Start plan generation
GET	/plan-runs/{run_id}	Generation status
GET	/plans/{id}	Plan overview
GET	/plans/{id}/progress	Progress summary
GET	/plans/{id}/next-action	Determine next actionable task
GET	/plans/{id}/events	SSE plan-run progress
POST	/tasks/{id}/complete	Complete task
POST	/tasks/{id}/reopen	Reopen task
GET	/plans/{id}/places	Plan places
POST	/plans/{id}/route/recalculate	Recalculate route

15.3 Request flow

HTTP request
-> route dependency: verify Firebase token
-> load authenticated user
-> validate Pydantic request
-> call service
-> repository / agent orchestration
-> validate output
-> persist transaction
-> map to response schema
-> return response

16. AI Agent Architecture and Tool Contracts

16.1 Agent responsibilities

Agent	Input	Output
Activity Understanding	Raw intent + user context	Structured activity context + missing fields + assumptions
Research	Activity context + questions	Current facts + evidence candidates
Maps	Origin/destination/constraints	Route + distance/time + route candidates
Recommendations	Context + place candidates	Ranked contextual suggestions with reasons/evidence
Planner	Context + facts + routes	Sections, tasks, dependencies, packing, itinerary
Verifier	Draft plan + evidence + constraints	Validation report + corrected draft / warnings

16.2 Narrow tool contracts

```
compute_route(origin, destination, mode, departure_time?)
compute_route_matrix(origins[], destinations[], mode, departure_time?)
search_places(text_query, location_bias?, type_filter?)
search_nearby(location, types[], radius_meters)
get_place_details(provider_place_id, fields[])
search_along_route(route_polyline, query, corridor_width_meters)
search_web(query, recency?, domain_filter?)
```

Tools should return structured provider-normalized results. Tool wrappers should validate inputs, enforce field masks, record latency/status and expose only the fields that the planner actually needs.

16.3 Orchestrator rules

- Limit tool-call count and recursion depth.
- Keep a deterministic top-level workflow for the MVP.
- Persist a plan run even when generation fails.
- Verify structured output before database writes.
- Never expose raw SQL or arbitrary code execution as an agent tool.

17. Grounding, Evidence and Verification

17.1 Trust hierarchy

1. Explicit user-provided facts
2. Structured first-party provider data (Maps / Places)
3. Current search-grounded sources
4. Model inference / suggestions
5. Unknown - ask or omit

17.2 Evidence object

Every important current fact should be traceable to an evidence record where feasible. The evidence layer should capture source type, source URI, title, retrieval time, a concise claim and metadata. This supports user trust and debugging.

17.3 Claim classes

Claim	Example	Handling
User fact	"My appointment is at 10 AM."	Store as user-provided context.
Provider fact	Route distance / place address	Store provider source and retrieval time.
Current external fact	Opening hours / event time	Ground and timestamp; may expire.
Inference	"Carry a rain jacket because weather may change."	Mark as recommendation, not fact.
Unknown	Institution-specific document rule not found	Do not invent; ask user or state uncertainty.

17.4 Verification checklist

- Schema valid?
- Required context satisfied?
- No impossible dates/order?
- No circular task dependencies?
- No unsupported critical claims?
- Provider data matches requested location?
- Tool results are not stale beyond the defined TTL for the use case?
- Plan is safe to persist?

18. Security and Trust Boundaries

18.1 Authentication

Client authenticates with Firebase. Backend receives the Firebase ID token over HTTPS and verifies it with the Firebase Admin SDK before establishing application identity. (R11)

18.2 Authorization

Every activity, plan, task and related resource must be scoped to the authenticated user. Never trust an object ID alone. Service methods should verify ownership or authorization before access.

18.3 AI security

Current Google security guidance for ADK highlights that agents become a distinct security problem when connected to live systems. LIFEOS should therefore assume prompt injection, malicious tool arguments and unintended state mutation are possible. (R15)

Control	Rule
Tool allowlist	Each agent sees only the tools it needs.
Read/write separation	MVP planning tools are read-oriented. Side effects need explicit application approval.
Input validation	Validate both user input and model-produced tool arguments.
Output validation	Validate structured model output before persistence.
Secrets	Never commit API keys/passwords. Use environment config locally and Secret Manager in deployment.
Logging hygiene	Do not log raw auth tokens or unnecessary sensitive content.
Rate limits	Protect generation endpoints from abuse and runaway model/tool usage.

18.4 External action policy

Booking, sending messages, making payments or modifying external records should be represented as a separate approval-controlled capability, not a default consequence of plan generation.

19. Reliability, Errors, Retries and Idempotency

19.1 Error taxonomy

Class	Examples	Action
VALIDATION	Bad request, invalid dates	4xx, no retry
AUTH	Invalid/expired token	401/403, no retry
NOT_FOUND	Missing plan	404, no retry
PROVIDER_TRANSIENT	Timeout, 429, temporary 5xx	Bounded exponential retry
AI_OUTPUT_INVALID	Schema mismatch	Repair / bounded retry / fail run
CONFLICT	Stale update / duplicate request	409 or idempotent reuse
INTERNAL	Unexpected bug	500, log with trace ID

19.2 Idempotency

Generation endpoints should accept an idempotency key or a client-generated request ID. Persist the key against a plan run so a mobile retry does not create duplicate generation jobs.

19.3 Transactions

Persist a verified plan inside a transaction boundary. If sections or tasks fail validation, do not leave a half-generated plan marked as ready.

19.4 Retry budget

Use bounded retries for network calls. Do not recursively retry model/tool failures indefinitely. Record each attempt in `tool_calls` or run metrics.

20. Testing and Evaluation Strategy

20.1 Testing layers

Layer	What to test
Unit	Services, validation, dependency logic, next-action calculation, error mapping.
Repository integration	CRUD, joins, constraints, transactions, migrations.
API integration	Auth, endpoint contracts, HTTP status codes, ownership boundaries.
Provider integration	Maps/Places/Search adapters with mocked and selected live checks.
Agent evaluation	Scenario dataset for activity understanding, planning, grounding and recovery.
End-to-end	Natural-language activity -> generated plan -> user edits -> completion -> next action.

20.2 Evaluation dataset

Build a golden set of at least 30 diverse scenarios before tuning the agent. Suggested categories: travel, appointment, college/institution visit, interview, relocation, shopping, event, conference and generic custom activity.

20.3 Metrics

Metric	What it measures	Initial target
Schema validity	Plan output matches required structure	$\geq 95\%$
Critical unsupported-claim rate	Important facts produced without evidence	$< 5\%$
Task actionability	Human reviewers can execute tasks without rewriting	$\geq 90\%$
Dependency validity	Task graph contains no illegal cycles	100%
Route integrity	Route endpoints/mode match requested context	100% in test cases
Failure recovery	Transient provider failure handled without corrupt state	100% in integration cases

Targets above are engineering goals for this project, not claims about current system performance.

21. Observability and Operations

21.1 Correlation identifiers

```
request_id
user_id (internal)
activity_id
plan_id
plan_run_id
tool_call_id
```

Every significant log event should carry the smallest set of correlation IDs needed to trace a failure from HTTP request to service to agent to provider tool call.

21.2 Structured logs

Event	Fields
HTTP request	request_id, method, path, status, latency_ms
Plan run	run_id, activity_id, model, status, duration_ms
Tool call	run_id, tool_name, provider, status, latency_ms
Persistence	entity, operation, duration_ms, outcome
Error	error_code, trace ID, safe message, context IDs

21.3 Operational dashboards

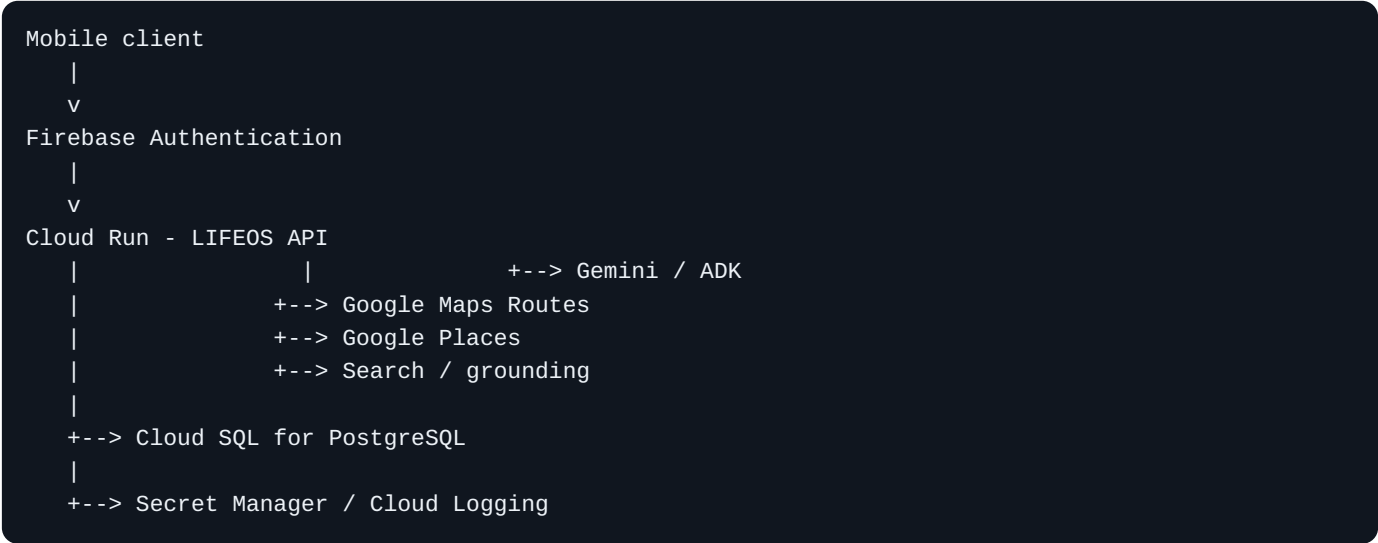
Track API latency/error rate, generation success rate, average tool calls per run, provider failure rate, model token usage where available, DB connection errors and active plan runs.

21.4 Run replay / debugging

Store prompt version, model name, high-level inputs and structured outputs/validation results needed for debugging, while applying a privacy policy to raw user content and provider payloads.

22. Deployment Plan

22.1 Target topology



22.2 Local vs cloud

Concern	Local	Cloud
API	uvicorn dev server	Cloud Run container
DB	PostgreSQL 16 local	Cloud SQL PostgreSQL
Secrets	.env, gitignored	Secret Manager / env integration
Auth	Firebase project credentials	Firebase + service account
Migrations	Alembic CLI	Controlled migration step in deployment pipeline

22.3 Cloud Run design implications

Cloud Run requests have a configurable timeout up to 60 minutes, but the product should not depend on a long-lived HTTP request for complex generation. Use a persisted plan-run record and asynchronous progress updates. (R12)

22.4 Deployment checklist

- Dockerfile builds reproducibly.
- Container listens on `0.0.0.0:$PORT`.
- Health endpoint is available.
- Secrets are not baked into the image.
- Database migrations are applied by a controlled step.
- Cloud Run service account has only required IAM roles.

23. Development Roadmap

- **Phase 0 - Freeze design**

Finalize ER model, enums, API contracts, agent schemas and edge-case decisions.

- **Phase 1 - Database**

Implement SQLAlchemy models, import model registry, generate first Alembic migration, apply and inspect schema.

- **Phase 2 - Core application layer**

Dependency injection, errors, authentication, common schemas, repository patterns and service boundaries.

- **Phase 3 - Activity vertical slice**

`POST /activities` -> persist intent -> `GET /activities/{id}`.

- **Phase 4 - First AI slice**

Activity Understanding -> structured context -> Plan Service -> basic sections/tasks -> persistence.

- **Phase 5 - Task system**

Task completion, subtasks, dependencies, progress and next-action calculation.

- **Phase 6 - Maps and places**

Routes, route stops, place search/details, contextual recommendation storage.

- **Phase 7 - Research and verification**

Evidence model, search grounding, verifier agent, uncertainty handling.

- **Phase 8 - Async plan runs**

Run state machine, SSE progress, retry/idempotency behavior.

- **Phase 9 - Testing/evaluation**

Unit/integration suite plus golden scenario dataset and regression tests.

- **Phase 10 - Deployment**

Docker, Cloud Run, Cloud SQL, secrets, observability and CI/CD.

Recommended implementation strategy: ship one complete vertical slice before building the full multi-agent system.

24. Definition of Ready / Done

24.1 Definition of Ready - before coding an entity or feature

☐	Purpose of the feature is explicit.
☐	Inputs and outputs are defined.
☐	Database ownership and relationships are decided.
☐	Validation and error cases are listed.
☐	Authorization boundary is clear.
☐	External provider dependency is identified.
☐	Test cases exist conceptually before implementation.

24.2 Definition of Done - for a backend feature

☐	Model/schema implemented.
☐	Migration created and tested.
☐	Service logic implemented with boundaries.
☐	Repository operations tested.
☐	Routes return documented status codes.
☐	Authentication and ownership checks applied.
☐	Error handling and logs implemented.
☐	Unit/integration tests pass.
☐	Agent feature has an evaluation case where relevant.
☐	No secrets or provider keys committed.

25. Pre-Development Checklist

A. Product

☒	Core product statement frozen.
☒	MVP scope and out-of-scope behavior documented.
☐	Primary demo scenario selected and scripted.

B. Architecture

☒	Modular monolith chosen.
☒	Agent/tool/application boundaries defined.
☐	Async plan-run strategy implemented and tested.

C. Database

☒	PostgreSQL database and user created.
☒	SQLAlchemy async connection verified.
☒	Alembic connected to PostgreSQL.
☒	Conceptual ER model drafted.
☐	SQLAlchemy models finalized.
☐	First migration generated and applied.

D. AI

☒	Model/agent stack researched.
☐	Structured output schemas frozen.
☐	Agent instructions/prompts versioned.
☐	Golden evaluation dataset created.

E. Integrations

☒	Routes/Places capabilities researched.
☐	API credentials and project configuration created.
☐	Provider adapters implemented behind integration interfaces.

F. Security

☒	Authentication architecture selected.
☐	Firebase token verification implemented.
☐	Tool allowlists and approval policy encoded.
☐	Secret management finalized for cloud deployment.

26. Open Decisions and Future Expansion

26.1 Decisions to make before the first feature merge

Decision	Recommended baseline	Reason
UUID strategy	Application-generated UUIDs	Stable public identifiers and good PostgreSQL fit.
Soft delete	Use status/archive where useful; avoid blanket soft delete	Simpler semantics.
Plan regeneration	New plan snapshot + new plan run	Preserves history and avoids partial overwrite.
Flexible AI context	JSONB for evolving context only	Balances schema stability with AI variability.
Agent writes	Through application services only	Centralizes validation and authorization.
Async execution	PlanRun state machine + background workflow	Supports long operations without tying up HTTP.

26.2 Future extraction candidates

Only extract a service when there is a concrete reason: independent scaling, independent deployment, different runtime needs or team ownership. The first likely candidate would be an AI/agent worker, followed by specialized integrations only if load or reliability requirements demand it.

26.3 Future features

- Collaborative plans and shared activities
- Calendar synchronization
- Approval-gated bookings and messaging
- Personal preference profiles
- Memory across activities
- Advanced route optimization and alternative modes
- Mobile notifications / reminders
- Plan templates and reusable activity patterns

Keep future features out of the initial schema unless they have a current ownership/relationship need.
Design for extension, not for speculative completeness.

27. Reference Sources

The technical research sections were checked against current first-party documentation available around 28 Sep 2026. Provider capabilities and pricing can change, so re-check the linked sources immediately before integration/deployment.

1. **R1 - Gemini API Models** - Google AI for Developers. Stable Gemini model catalog; includes Gemini 3.8 Flash. <https://ai.google.dev/gemini-api/docs/models>
2. **R2 - Gemini 3.8 Flash** - Google AI for Developers. Capabilities, model ID and current release details. <https://ai.google.dev/gemini-api/docs/models/gemini-3.8-flash>
3. **R3 - Interactions API** - Google AI for Developers. GA status and agent/model interface. <https://ai.google.dev/gemini-api/docs/interactions-overview>
4. **R4 - ADK Documentation / Development** - Google. Current ADK development, evaluation and deployment guidance. <https://google.github.io/adk-docs/>
5. **R5 - Multi-agent patterns in ADK** - Google Developers Blog. Sequential pipeline and orchestration patterns. <https://developers.googleblog.com/developers-guide-to-multi-agent-patterns-in-adk/>
6. **R6 - Routes API: Compute Routes** - Google Maps Platform. Route generation and field masks. https://developers.google.com/maps/documentation/routes/compute_route_directions
7. **R7 - Routes API: Compute Route Matrix** - Google Maps Platform. Origin/destination matrix calculations. <https://developers.google.com/maps/documentation/routes/reference/rest/v2/TopLevel/computeRouteMatrix>
8. **R8 - Routes API Overview** - Google Maps Platform. Compute Routes and Compute Route Matrix. <https://developers.google.com/maps/documentation/routes/reference/rpc>
9. **R9 - Places API (New) Overview** - Google Maps Platform. Search, details, photos and current API generation. <https://developers.google.com/maps/documentation/places/web-service/overview>
10. **R10 - Places Search and Field Masks** - Google Maps Platform. Text Search / Nearby Search requirements. <https://developers.google.com/maps/documentation/places/web-service/data-fields>
11. **R11 - Firebase Verify ID Tokens** - Firebase Documentation. Custom backend token verification pattern. <https://firebase.google.com/docs/auth/admin/verify-id-tokens>
12. **R12 - Cloud Run Autoscaling** - Google Cloud. Default scaling behavior. <https://docs.cloud.google.com/run/docs/about-instance-autoscaling>
13. **R13 - Cloud Run Request Timeout** - Google Cloud. Request timeout limits. <https://docs.cloud.google.com/run/docs/configuring/request-timeout>
14. **R14 - Cloud Run + Cloud SQL PostgreSQL** - Google Cloud. Connection patterns. <https://docs.cloud.google.com/sql/docs/postgres/connect-run>
15. **R15 - Zero-trust AI agents with ADK** - Google Developers Blog, Aug 2026. Security implications of agent access to live systems. <https://developers.googleblog.com/build-zero-trust-ai-agents-with-googles-agent-development-kit/>

Document note

This is a planning baseline, not a guarantee that every future implementation detail will remain unchanged. Changes to APIs, provider policies, pricing or model availability should trigger an update to the technical decision log before implementation is adjusted.